

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Образовательная программа Мобильные и сетевые технологии
Направление подготовки 09.03.03 Прикладная информатика

О Т Ч Е Т
по курсовой работе
Тема «Разработка Android-приложения "Мессенджер" с использованием
архитектуры MVVM, навигации и интеграции с API»

Обучающийся:
Зотов Михаил Дмитриевич, К3441

Преподаватель:
Шатинский Григорий Сергеевич

Санкт-Петербург
2026

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ.....	2
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ.....	3
ВВЕДЕНИЕ.....	4
1 Анализ предметной области.....	5
1.1 Обзор и анализ аналогичных решений.....	5
1.2 Функциональные требования.....	7
1.3 Нефункциональные требования.....	8
1.4 Обоснование выбора средств разработки.....	9
2 Проектирование.....	10
2.1 Архитектура системы (С4-модель).....	11
2.2 Структура базы данных (ER-диаграмма).....	13
2.3 Проектирование структуры базы данных.....	13
2.4 Пример AS-IS (как сейчас) / TO-BE (как должно быть).....	16
2.4.1 Модель AS-IS (Текущее состояние).....	16
2.4.2 Модель TO-BE (Целевое состояние).....	16
2.4.3 Обоснование проектных решений.....	19
ЗАКЛЮЧЕНИЕ.....	20
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	22

ВВЕДЕНИЕ

Курсовая работа посвящена разработке простого Android-приложения "Мессенджер", которое демонстрирует использование современных инструментов Android-разработки. Приложение включает в себя экраны ленты сообщений, профиля пользователя и настроек, с интеграцией данных из внешнего API, локальным хранением, навигацией и фоновыми задачами.

Цель работы — реализовать приложение в соответствии с требованиями четырех лабораторных работ (ЛР1–ЛР4), используя архитектуру MVVM, Jetpack Compose для UI, Retrofit для сетевых запросов, Room для базы данных, WorkManager для фоновых задач и другие компоненты Android Jetpack.

В ходе работы были выполнены следующие задачи:

- Сбор требований и настройка проекта;
- Реализация навигации и базового UI;
- Управление состоянием с использованием ViewModel и LiveData;
- Интеграция с API и оффлайн-хранение данных;
- Улучшение интерфейса, добавление уведомлений и фоновой синхронизации.

Стек технологий: Kotlin, Android Jetpack (Compose, Navigation, ViewModel, LiveData, Room, Retrofit, Coroutines, WorkManager), Material Design.

Данные для ленты сообщений загружаются из публичного API (JSONPlaceholder). Приложение поддерживает оффлайн-режим, лайки сообщений, смену темы и уведомления о новых данных.

1 Обзор проекта и используемых технологий

Проект "Мессенджер" представляет собой простое Android-приложение, ориентированное на демонстрацию ключевых аспектов мобильной разработки: от базовой структуры до интеграции с внешними сервисами. Приложение состоит из одной главной активности (MainActivity), которая служит точкой входа и контейнером для навигации. Внутри реализована bottom-навигация с тремя экранами: лента сообщений (Feed), профиль (Profile) и настройки (Settings).

Для UI использован Jetpack Compose, что позволяет создавать декларативный интерфейс без XML-элементов. Это упрощает управление состоянием и анимациями. Навигация построена на NavController и Navigation Graph, обеспечивая плавные переходы между экранами. Отслеживание жизненного цикла компонентов (Activity, ViewModel) фиксируется в логах для отладки.

Настройка зависимостей показана в листинге 1

Листинг 1 – Фрагмент build.gradle.kts (настройка зависимостей)

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
    alias(libs.plugins.kotlin.compose)  
    kotlin("kapt")  
}  
  
android {  
    namespace = "com.waycooler.messengermih"  
    compileSdk = 36  
    defaultConfig {  
        applicationId = "com.waycooler.messengermih"  
        minSdk = 24  
        targetSdk = 36  
        versionCode = 1  
    }  
}
```

```

        versionName = "1.0"
    }
    buildFeatures {
        compose = true
    }
}
dependencies {
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    implementation(libs.androidx.activity.compose)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.compose.ui)
    // ... (остальные зависимости, включая Retrofit, Room,
    WorkManager)
}

```

Проект настроен на игнорирование ненужных файлов в .gitignore, а APK-файл добавлен для быстрой проверки. При запуске приложение запрашивает разрешения на уведомления и интернет.

2 Архитектура приложения

2.1 MVVM и управление состоянием

Архитектура приложения построена на паттерне MVVM, который разделяет ответственность: View (UI в Compose) отображает данные, ViewModel управляет состоянием, Model (Repository) работает с данными. Это обеспечивает устойчивость к изменениям конфигурации (например, повороту экрана) и реактивные обновления через LiveData.

Для экрана профиля создана ProfileViewModel, хранящая имя и статус пользователя. Код которого показан в листинге 2. Данные редактируются через TextField и сохраняются во ViewModel. Для настроек — SettingsViewModel с переключателем тем (светлая/темная), состояние которого сохраняется в DataStore для персистентности.

Листинг 2 – Фрагмент ProfileViewModel.kt

```
class ProfileViewModel : ViewModel() {  
    private val _name = MutableLiveData("Имя  
пользователя")  
    val name: LiveData<String> = _name  
    private val _status = MutableLiveData("Мой  
статус")  
    val status: LiveData<String> = _status  
    init {  
        Log.d("ProfileViewModel", "init")  
    }  
    override fun onCleared() {  
        super.onCleared()  
        Log.d("ProfileViewModel", "onCleared")  
    }  
    fun updateName(value: String) {  
        _name.value = value  
    }  
    fun updateStatus(value: String) {  
        _status.value = value  
    }  
}
```

При повороте экрана данные не теряются благодаря ViewModel. Логи жизненного цикла фиксируют этапы `init` и `onCleared`. UI обновляется реактивно: в Compose используется `observeAsState` для привязки LiveData к компонентам. Динамика состояния в ViewModel отображена в таблице 1.

Таблица 1 – Динамика состояния в ViewModel (схематично)

Событие	Лог	Действие
Создание ViewModel	init	Загрузка начальных данных
Обновление данных	-	Изменение LiveData
Уничтожение	onCleared	Очистка ресурсов

2.2 Навигация и пользовательский интерфейс

Навигация реализована через BottomNavigation и NavHost в главной активности. Код показан в листинге 3. Каждая вкладка открывает composable-экран: Feed с заглушкой, Profile с редактируемыми полями, Settings с переключателем. Переходы управляются NavController, что обеспечивает single-top и попап до старта.

Интерфейс улучшен элементами Material Design: Card для сообщений, FloatingActionButton для обновления, Switch для тем. Лайки сообщений меняют иконку при клике, состояние сохраняется в БД. Интерфейс показан на рисунке 1.

Листинг 3 – Фрагмент MessengerMihApp.kt (навигация)

```
@Composable
fun MessengerMihApp() {
    val settingsViewModel: SettingsViewModel =
viewModel()

    val darkTheme =
settingsViewModel.darkTheme.observeAsState(false).value

    MessengerMihTheme(darkTheme = darkTheme) {
        val navController = rememberNavController()
        Scaffold(
            bottomBar = {
```

```

        NavigationBar {
            // ... (элементы навигации)
        }
    }

    ) { innerPadding ->
        NavHost (
            navController = navController,
            startDestination =
Screen.Feed.route,
            modifier =
Modifier.padding(innerPadding)
        ) {
        }
    }
}
}
}

```

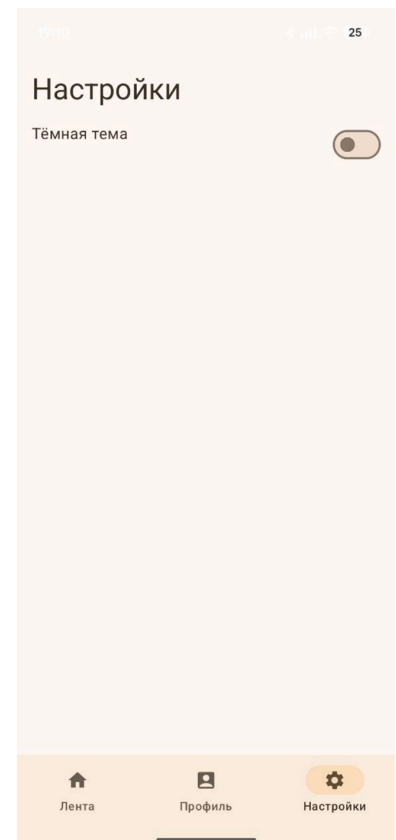
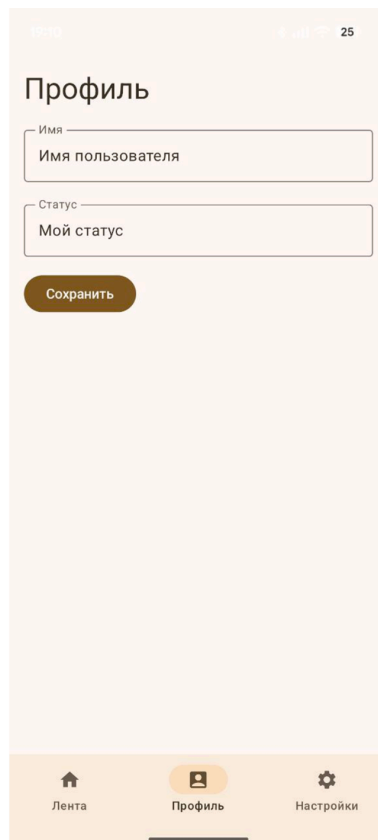


Рисунок 1 – Интерфейс

3 Интеграция данных и оффлайн-доступ

3.1 Работа с API и сетевыми запросами

Данные для ленты сообщений загружаются из публичного API через Retrofit с корутинами. Создан MessageRepository как единая точка доступа: он получает данные асинхронно, преобразует в Entity и сохраняет в Room. При отсутствии сети используются локальные данные. Код MessageRepository показан в листинге 4.

Листинг 4 – Фрагмент MessageRepository.kt

```
class MessageRepository(context: Context) {  
    private val messageDao =  
AppDatabase.getInstance(context).messageDao()  
    private val api = RetrofitClient.api  
    suspend fun getMessages(): List<MessageEntity>  
= withContext(Dispatchers.IO) {  
        try {  
            val remoteMessages = api.getMessages()  
            // ... (преобразование и вставка в БД)  
            entities  
        } catch (e: Exception) {  
            messageDao.getAllMessages()  
        }  
    }  
}
```

Сетевые операции выполняются только в корутинах, ViewModel (FeedViewModel) загружает данные в viewModelScope. Пример показан в листинге 5. Кнопка "Обновить" (FAB) вызывает ручное обновление.

Листинг 5 – Фрагмент FeedViewModel.kt

```
class FeedViewModel(application: Application) :  
AndroidViewModel(application) {
```

```

        private val repository =
MessageRepository(application)

        private val _messages =
MutableLiveData<List<MessageEntity>>()

        val messages: LiveData<List<MessageEntity>> =
_messages

        fun loadMessages() {
            viewModelScope.launch {

                _messages.value =
repository.getMessages()

            }

        }
    }
}

```

3.2 Локальное хранение в Room

Room используется для оффлайн-доступа: Entity для сообщений, DAO для операций (insert, getAll, toggleLike). Репозиторий интегрирует API и БД, обеспечивая единый интерфейс. Пример таблицы сообщений из БД показан в таблице 2.

Таблица 2 – Пример таблицы сообщений из БД

ID	Author	Text	isLiked
1	Пользователь 1	Текст сообщения	false
2	Пользователь 2	Другой текст	true

4 Фоновые задачи, уведомления и улучшение интерфейса

Фоновые обновления реализованы через WorkManager: периодическая задача (каждые 15 мин) обновляет данные при наличии сети и показывает уведомление "Новые данные получены". Код показан в листинге 6. Разрешения запрашиваются при запуске.

Листинг 6 – Фрагмент WorkManagerHelper.kt

```

object WorkManagerHelper {

    fun startSync(context: Context) {

```

```

                                val workRequest =
PeriodicWorkRequestBuilder<SyncMessagesWorker>(15,
TimeUnit.MINUTES)

.setConstraints(Constraints.Builder().setRequiredNetwo
rkType(NetworkType.CONNECTED).build())
                                .build()

WorkManager.getInstance(context).enqueueUniquePeriodic
Work("sync_messages", ExistingPeriodicWorkPolicy.KEEP,
workRequest)
        }
    }

```

Уведомления работают в фоне через NotificationCompat. Интерфейс улучшен: кастомный адаптер для списка (в Compose – LazyColumn с Card), лайки с изменением иконки.

Приложение реагирует на онлайн/оффлайн: репозиторий переключается на локальные данные.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы было разработано Android-приложение «Мессенджер», демонстрирующее практическое применение современных подходов и инструментов мобильной разработки. В процессе работы последовательно реализованы требования четырех лабораторных работ (ЛР1–ЛР4), что позволило пройти полный цикл создания приложения — от инициализации проекта и проектирования архитектуры до интеграции внешних сервисов и фоновых задач.

Приложение построено с использованием архитектурного паттерна MVVM, который обеспечил четкое разделение ответственности между пользовательским интерфейсом, логикой управления состоянием и слоем данных. Применение ViewModel и LiveData позволило корректно обрабатывать изменения жизненного цикла компонентов и сохранять состояние при изменении конфигурации устройства. Использование Jetpack Compose упростило разработку интерфейса, сделав его декларативным, реактивным и легко расширяемым.

В рамках работы реализована навигация между экранами с помощью Navigation Component, а также bottom-навигация, обеспечивающая удобство взаимодействия пользователя с приложением. Интеграция с внешним API через Retrofit и корутины позволила организовать асинхронную загрузку данных, а использование базы данных Room обеспечило оффлайн-доступ и локальное хранение информации. Репозиторий выступает единым источником данных, автоматически переключаясь между сетевым и локальным источником в зависимости от доступности интернета.

Дополнительно в приложении реализованы фоновые задачи с использованием WorkManager, которые выполняют периодическую синхронизацию данных и отправляют уведомления пользователю. Это позволило повысить автономность и удобство приложения. Улучшения пользовательского интерфейса, такие как лайки сообщений, смена темы

оформления и реактивные обновления контента, повысили интерактивность и соответствие принципам Material Design.

В результате выполнения курсовой работы были получены практические навыки разработки Android-приложений с использованием Android Jetpack, работы с асинхронными операциями, локальным хранением данных и архитектурными шаблонами. Разработанное приложение является расширяемым и может быть доработано в дальнейшем. В качестве возможных направлений развития можно выделить добавление системы аутентификации пользователей, реализацию обмена сообщениями в реальном времени с использованием WebSocket, а также улучшение безопасности и оптимизацию работы с данными.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers: сайт. URL: <https://developer.android.com/> (дата обращения: 01.10.2025)
2. Jetpack Compose Documentation: сайт. URL: <https://developer.android.com/jetpack/compose> (дата обращения: 01.10.2025)
3. Retrofit Documentation: сайт. URL: <https://square.github.io/retrofit/> (дата обращения: 30.10.2025)
4. Room Documentation: сайт. URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 30.10.2025)
5. WorkManager Documentation: сайт. URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 09.11.2025)
6. JSONPlaceholder. URL: <https://jsonplaceholder.typicode.com/> (дата обращения: 21.12.2025).